

# Guess Market - Exercise 1

**Name:** Dan Alushe

**ID:** 212472864

**Email:** danalushe15@gmail.com

**GitHub:** <https://github.com/danal15/guess-market-ex1>

## Bonus implemented

Save & Load system state. Commands 6 and 7 in the menu save the whole current state of the system to a file and load it back later (including closed events, trade history, balances and commissions). It is done with Java serialization of the Market object. The user enters a path without an extension and the program adds ".gmstate" to it.

## How to run

- Put engine.jar, console.jar and run.bat in the same folder and double click run.bat (or run: java -jar console.jar).
- Choose commands by typing their number in the menu.
- Command 1 asks for a full path to an XML file. Spaces in the path are fine, quotes are also fine.
- Commands 2-5 only work after a file was loaded.
- Commands 6-7 (the bonus) save/load the whole system to a file. Enter the path without an extension, the program adds ".gmstate" by itself.

## Project structure

There are two modules like the exercise required:

- engine - the system engine. Loads and validates the XML, does the LMSR calculations, and manages the events, accounts and trade history. It has no prints and no scanner.
- console - the UI. Holds main(), the menu loop and all the input/output. It only talks to the engine through the GMEngine interface and gets DTO objects back, so it never touches the engine's internal objects.

Main classes: Event (an event with its two options, account and trade history, with the buy and close logic), Market (holds all the loaded events, this is also the object that gets serialized in the bonus), MarketFileLoader (XML reading + validation), LmsrMath (the price/cost formulas), GuessMarketEngine (implements GMEngine and connects everything), and in the console module: ConsoleUI, ConsoleInput, MenuCommand, Main.

## Decisions I made

- Event numbers in the menus are the position in the list (1,2,3...), not the id from the XML, because the ids are just unique numbers and don't have to start from 1.
- In commands 4/5 the list of active events is renumbered from 1.
- In the trade history, "price paid" is the price of the shares without the commission. The commission is shown separately.
- On-close commission is calculated on the winners' payout (shares \* 1\$), like in the simulation file from the course site.
- The XML is read with the JDK's built-in DOM parser, so there are no external jars.
- Following the clarification in the course forum, the event account is not reset when the event closes. The account starts at 0 when the file is loaded, money from purchases and commissions goes in, the winners' payment goes out, and whatever is left stays there (it can be negative - that is what the MM paid to subsidize the event).

- The exceptions are unchecked (RuntimeException), and the UI catches everything around each command so the program never crashes on bad input or a bad file.
- The bonus is done with Java serialization: all the model classes implement Serializable, so saving is just writing the Market object to a file with ObjectOutputStream, and loading reads it back.